

Algorithm Analysis & Design

Tutorial: Huffman Coding & Dynamic Programming

Kushal Mangla

Semester 7 — 2026

Part I — Huffman Coding

- ① Huffman Coding Goal
- ② Prefix-Free Codes
- ③ Prefix Code $\leftrightarrow$ Binary Tree
- ④ Building the Huffman Tree
- ⑤ Huffman Algorithm
- ⑥ Worked Example

Part II — Dynamic Programming

- ⑦ DP Core Framework
- ⑧ DP Template
- ⑨ Grid Paths (CSES) — Intro DP
- ⑩ Grid Paths: Walkthrough
- ⑪ 0/1 Knapsack
- ⑫ 0/1 Knapsack: 1D Optimisation
- ⑬ DP Counting: Two Big Questions
- ⑭ 0/1 Combination
- ⑮ Combination Sum (Unbounded)
- ⑯ Permutation / Order Matters
- ⑰ The Loop-Order Trick
- ⑱ Complete Variation Framework

Slide 1 — Huffman Coding

Goal. Given symbols with frequencies f_i , assign binary codes so that the total cost is minimised:

$$\text{Total cost} = \sum_i f_i \ell_i$$

where ℓ_i = length of code for symbol i .

Key idea:

- Frequent symbols $\longrightarrow$ **short codes**
- Rare symbols $\longrightarrow$ **long codes**

This is exactly the *weighted tree objective* from the lecture.

Slide 2 — Prefix-Free Codes

A code is **prefix-free** if:

No codeword is a prefix of another codeword.

✓ **Valid:**

A = 0
B = 10
C = 110
D = 111

✗ **Invalid:**

A = 0
B = 01

0 is a prefix of 01.

No codeword is a prefix of another.

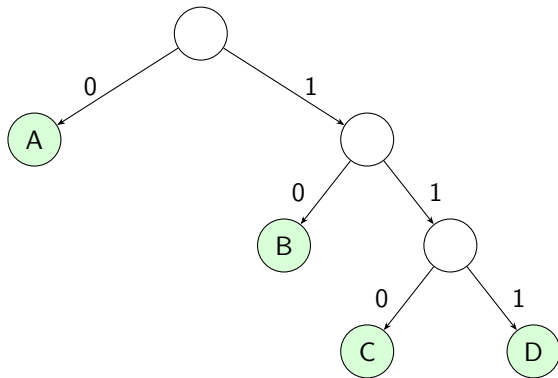
Why do we need this? It allows **unique decoding**.

0 | 110 | 111 $\longrightarrow$ A C D

The stream 0110111 can be decoded unambiguously.

Slide 3 — Prefix Code $\leftrightarrow$ Binary Tree

A prefix-free binary code can be represented by a **binary tree**:



Prefix-free code $\iff$ symbols are placed at **leaves** of a binary tree

If a symbol were at an internal node, its code would be a prefix of a descendant's code.

Slide 4 — When Can We Form a Huffman Tree?

For n symbols with positive frequencies:

- 1 Start with n leaf nodes.
- 2 Repeatedly combine the **two smallest frequencies**.
- 3 The combined node becomes their parent.
- 4 Continue until one root remains.

The resulting tree is a **full binary tree**: every internal node has exactly 2 children.

Not every binary tree is optimal.

Huffman specifically constructs the tree minimising $\sum_i f_i \ell_i$

Slide 5 — Huffman Algorithm

Algorithm

- 1 Put every symbol into a **min-heap** keyed by frequency.
- 2 Remove the two minimum-frequency nodes.
- 3 Create a new node with frequency = sum of the two.
- 4 Insert the new node back into the heap.
- 5 Repeat until only one node remains.
- 6 Assign 0 to left edges, 1 to right edges.

Complexity: $O(n \log n)$
 $O(n)$ heap operations, each $O(\log n)$

Slide 6 — Huffman Example

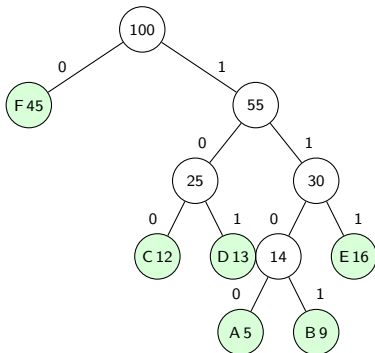
Frequencies:

Symbol	Freq
A	5
B	9
C	12
D	13
E	16
F	45

Merges:

- 1 $5 + 9 = 14$
- 2 $12 + 13 = 25$
- 3 $14 + 16 = 30$
- 4 $25 + 30 = 55$

Resulting tree:



Encoding:

Sym	Code
F	0
C	100
D	101
A	1100
B	1101
E	111

Observation:

F (highest freq)
→ shortest code

A, B (low freq)
→ longer codes

Slide 7 — DP: Core Framework

For **every** DP problem, ask:

① **What is the state?**

What information completely describes the remaining problem?

② **What are the choices?**

What can I do at this state?

③ **What is the transition?**

$$dp[\text{state}] = \text{best / count over choices}$$

④ **Base case?**

⑤ **Order of computation?**

⑥ **Complexity?**

Lecture's interval-scheduling and shortest-path examples follow exactly this pattern:
define an optimal subproblem → derive the recurrence → memoize.

Optimisation DP:

```
dp[state] = best answer

dp[state] = max/min(
    choice_1,
    choice_2,
    ...
);
```

Counting DP:

```
dp[state] = number of ways

dp[state] += dp[prev_state];
```

General Thought Process

STATE → CHOICES → TRANSITION → BASE CASE → ORDER →
COMPLEXITY

Slide 9 — Grid Paths (CSES) — Introductory DP

Problem. Given an $n \times n$ grid, count paths from $(1, 1)$ to (n, n) . You may only move **right** or **down**. Some cells are blocked (*). Answer modulo $10^9 + 7$.

Applying the DP framework:

- ① **State:** $dp[i][j]$ = number of paths from $(1, 1)$ to (i, j)
- ② **Choices:** arrive from *above* or from the *left*
- ③ **Transition:**

$$dp[i][j] = \begin{cases} 0 & \text{if cell } (i, j) \text{ is blocked} \\ dp[i-1][j] + dp[i][j-1] & \text{otherwise} \end{cases}$$

- ④ **Base case:** $dp[1][1] = 1$ (if not blocked)
- ⑤ **Order:** row by row, left to right
- ⑥ **Complexity:** $O(n^2)$

Answer: $dp[n][n]$

Slide 10 — Grid Paths: Walkthrough

Grid (4×4 , one trap):

S			
1	1	1	1
1	* 0	1	2
1	1	2	4
1	2	4	E 8

Answer: $dp[4][4] = 8$

Blocked cell $\Rightarrow dp = 0$, its effect propagates right & down.

Complete Code (C++):

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<string> grid(n);
    for (auto& row : grid) cin >> row;

    vector<vector<long long>> dp(
        n, vector<long long>(n, 0));
    const int MOD = 1e9 + 7;

    dp[0][0] = (grid[0][0] == '.') ? 1 : 0;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '*') {
                dp[i][j] = 0; continue;
            }
            if (i > 0) dp[i][j] += dp[i-1][j];
            if (j > 0) dp[i][j] += dp[i][j-1];
            dp[i][j] %= MOD;
        }
    }
}
```

Slide 11 — 0/1 Knapsack

Given: $\text{weight}[i]$, $\text{value}[i]$, capacity W . Each item taken **at most once**.

State:

$dp[i][w] = \text{max value using first } i \text{ items with capacity } w$

Choices:

- Don't take item i : $dp[i-1][w]$
- Take item i : $v_i + dp[i-1][w - w_i]$

$$dp[i][w] = \max\left(dp[i-1][w], v_i + dp[i-1][w - w_i]\right)$$

Complexity: $O(nW)$ (pseudo-polynomial)

Slide 12 — 0/1 Knapsack: 1D DP

Compress to 1D:

```
vector<int> dp(W + 1, 0);

for (int i = 0; i < n; i++) {
    for (int w = W; w >= weight[i]; w--) {
        dp[w] = max(dp[w],
                    value[i] + dp[w - weight[i]]);
    }
}
```

Why iterate backwards? Because item i must **not be reused**.

Iterating $w = W \rightarrow \text{weight}[i]$ ensures the current item doesn't affect another state that will use it again.

Mnemonic: 0/1 $\implies$ capacity loop **BACKWARD**

Slide 13 — DP Counting: The Two Big Questions

When counting ways, first ask:

Q1. Can I **reuse** an item?

Q2. Does **order** matter?

	Order DOES NOT matter	Order MATTERS
No reuse	0/1 Combination	0/1 Permutation
Reuse allowed	Combination Sum	Permutation / Unbounded

This is the most important classification for DP counting variations.

Slide 14 — 0/1 Combination (No Reuse, Order Doesn't Matter)

Let $dp[s]$ mean: the number of ways to make sum s using the items that we have processed so far.

Example: $nums = [1,2,3]$, $target = 4$

Each number used **once**.

Ways: $\{1 + 3\}$ only.

```
dp[0] = 1;

for (each item x) {
    for (s = target; s >= x; s--) {
        dp[s] += dp[s - x];
    }
}
```

When we iterate backward, the current item can only use DP states that existed BEFORE we started processing this item.

Slide 15 — Combination Sum (Reuse Allowed, Order Doesn't Matter)

Let $dp[s]$ mean: the number of ways to make sum s using the items that we have processed so far.

Example: $nums = [1,2,3]$, $target = 4$

Combinations: $\{1+1+1+1, 1+1+2, 1+3, 2+2\} \implies answer = 4$.

```
dp[0] = 1;

for (int x : nums) {
    for (int s = x; s <= target; s++) {
        dp[s] += dp[s - x];
    }
}
```

We give every combination a fixed ordering based on the order in which we process the items.

That's why the **items** loop sits on the **outside**.

Slide 16 — Permutation / Order Matters

Example: `nums = [1,2,3]`, `target = 4`

Now `1+3` and `3+1` are **different**.

Sequences: `{1111, 112, 121, 211, 22, 13, 31}` $\implies$ **answer = 7**.

```
dp[0] = 1;

for (int s = 1; s <= target; s++) {
    for (int x : nums) {
        if (s >= x)
            dp[s] += dp[s - x];
    }
}
```

Combination:

`item  $\rightarrow$  sum (outer  $\rightarrow$  inner)`

Permutation:

`sum  $\rightarrow$  item (outer  $\rightarrow$  inner)`

Slide 17 — The Loop-Order Trick ★

Combination

Order does **not** matter.

$1+2 = 2+1$ counted *once*.

```
for (item : items)
  for (sum = item;
      sum <= target; sum++)
    dp[sum] += dp[sum-item];
```

Permutation

Order **matters**.

$1+2$ and $2+1$ counted *separately*.

```
for (sum = 1;
     sum <= target; sum++)
  for (item : items)
    dp[sum] += dp[sum-item];
```

Items outside $\Rightarrow$ Combination

Sum outside $\Rightarrow$ Permutation

Slide 18 — Complete DP Variation Framework ★★★

Problem	Reuse?	Order?	Outer loop	Inner loop
0/1 Knapsack	✗	—	items	capacity ↓
0/1 Combination	✗	✗	items	sum ↓
Unbounded Combination	✓	✗	items	sum ↑
Permutation	✓	✓	sum	items

The Key Rules

No reuse $\implies$ backward sum/capacity

Reuse $\implies$ forward sum

Combination $\implies$ items first

Permutation $\implies$ sum first

Slide 19 — Why Loop Order Changes the Answer

Example: `nums = [1,2]`, `target = 3`

Combination:

- $\{1 + 2\}$ — **1 way**

```
for (x : nums)
  for (s = x; s <= target; s++)
```

Permutation:

- $\{1 + 2, 2 + 1\}$ — **2 ways**

```
for (s = 1; s <= target; s++)
  for (x : nums)
```

Intuition: The outer loop defines what is allowed to change first.

- **Items first** → establish ordering among item types → *combinations*.
- **Sum first** → at each position choose any item → *permutations*.

Slide 20 — Don't Mix These Problems

Question 1: Can an element be used more than once?

NO $\implies$ **Backward**
YES $\implies$ **Forward**

Question 2: Are $1 + 2$ and $2 + 1$ different?

NO $\implies$ **Combination**
YES $\implies$ **Permutation**

Question 3: Are we maximising/minimising or counting?

max/min $\implies$ `max()` / `min()`
count $\implies$ `+=`

Good Luck!

Algorithm Analysis & Design — Tutorial

Huffman Coding & Dynamic Programming